

fn score_candidates_constants

Michael Shoemate (@Shoeboxam)

May 20, 2026

This proof resides in “contrib” because it has not completed the vetting process.

Proves soundness of `score_candidates_constants` in `mod.rs` at commit `f5bb719` (outdated¹). `score_candidates_constants` returns a fractional decomposition of alpha and an upper bound on dataset size to protect against overflow.

1 Hoare Triple

Precondition

Compiler-verified

None

Caller-verified

None

Pseudocode

```
1 def score_candidates_constants(size: Optional[u64], alpha: f64) -> tuple[u64, u64, u64]:
2     if alpha < 0.0 or 1.0 < alpha:
3         return ValueError("alpha must be within [0, 1]")
4
5     alpha_num_exact, alpha_den_exact = RBig.try_from(alpha).into_parts()
6
7     if size is not None:
8         # choose the finest granularity that won't overflow
9         # must have that size * denom < MAX, so let denom = MAX // size
10        alpha_den_approx = u64.MAX.neg_inf_div(size)
11    else:
12        # default to an alpha granularity of .00001
13        u64.exact_int_cast(10_000)
14
15    if alpha_den_exact < UBig.from_(alpha_den_approx):
16        alpha_num = u64.try_from(alpha_num_exact.into_parts()[1])
17        alpha_den = u64.try_from(alpha_den_exact)
18    else:
19        # number = alpha * denom
20        alpha_num_approx = u64.round_cast(alpha * f64.round_cast(alpha_den_approx))
21        alpha_num, alpha_den = alpha_num_approx, alpha_den_approx
22
23    if size is not None:
24        size_limit = size
```

¹See new changes with `git diff f5bb719..5532fe1 rust/src/transformations/quantile_score_candidates/mod.rs`

```

25     else:
26         size_limit = u64.MAX.neg_inf_div(alpha_den)
27
28     assert alpha_num <= alpha_den #
29     size_limit.alerting_mul(alpha_den) #
30
31     return alpha_num, alpha_den, size_limit #

```

Postcondition

Theorem 1.1. When the function does not fail, it returns the following constants:

- `alpha_num`
- `alpha_den`
- `size_limit`

The constants follow the following properties:

1. $\alpha_{num}/\alpha_{den} \in [0, 1]$
2. $size_limit \cdot \alpha_{den} < 2^{64}$

An error is raised if these properties cannot be met.

Proof. While the code is complicated, all that is needed is to observe that the function does not return until line 31, and that the conditions are explicitly checked on lines 28 and 29.

Therefore, the postcondition holds. □